

LongSeeker / Context-ReAct

Elastic Context Orchestration for Long-Horizon Search Agents - 深度研究与工程 Know-how

从“上下文摘要”到“上下文动作空间”：让 Agent 学会主动塑形自己的 Working Memory

长程 Agent Context Management 的演进

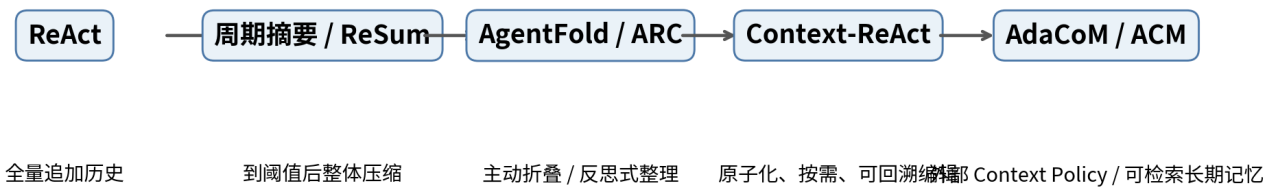


图1 长程 Agent Context Management 的演进

研究日期：2026-08-13 | 核心来源：LongSeeker arXiv:2605.05191 + 官方 GitHub 实现

目录

1. 一页结论：LongSeeker 到底解决什么
 2. Know-why：为什么长 Context Window 仍然不够
 3. Context-ReAct：把 Context Editing 升格为 Agent Action
 4. 五种原子操作与信息保真度
 5. Expressive Completeness：为什么 Compress 理论上已足够但工程上不够
 6. 数据合成与训练：如何教会 Agent 管 Context
 7. 推理 Runtime 与官方开源实现
 8. 实验结果：性能、Context Growth 与操作分布
 9. 为什么优于周期摘要 / Discard-all
 10. 与 ReSum / AgentFold / AdaCoM / ACM 的关系
 11. 可迁移 Know-how：生产级 Research Agent 怎么设计
 12. 局限、风险与改进方向
 13. 统一架构：Context Policy $\times$ Evidence State $\times$ Search Policy
 14. 结论
- 参考资料

1. 一页结论：LongSeeker 到底解决什么

LongSeeker 的关键贡献不是把 context window 做得更大，而是改变“历史如何进入下一步推理”的控制权。标准 ReAct 将每个 reasoning、tool call、observation 永久 append；ReSum 在阈值或周期点整体总结；Context-ReAct 则把 context manipulation 变成每一步都可以调用的 meta-actions，并与下一次标准工具调用共同生成。

- 核心范式： $S_i = (\text{reasoning}, \text{meta-operations}, \text{standard tool call}, \text{observation})$ 。下一步实际看到的是经过 meta-operations 变换后的 H' ，而不是原始 H 。
- 五个原子动作：Skip、Compress、Rollback、Snippet、Delete。它们分别对应保留、抽象、回退、精确保真、彻底删除。
- 训练：用 DeepSeek V3.2 作为 teacher，为 10k OpenSeeker 多跳问题生成带 context management 决策的轨迹，再对 Qwen3-30B-A3B 做 SFT。
- 结果：BrowseComp 61.5、BrowseComp-ZH 62.5、xbench-2505 78.0、GAIA-text 77.7。
- 最关键的系统证据：在最长 300-step 轨迹中，working context 平均稳定在约 15k tokens，而不是随步骤线性膨胀。

Context-ReAct：上下文编辑与工具调用进入同一个 Agent Policy

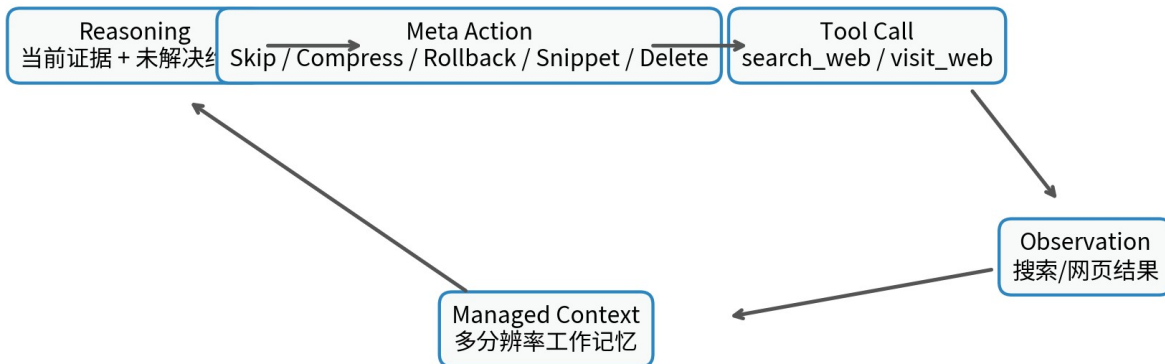


图2 Context-ReAct 将 Context 管理纳入同一个 Agent Policy

2. Know-why：为什么长 Context Window 仍然不够

“模型有 128k/256k context，所以不需要做 Context Engineering”是一个常见误区。LongSeeker 研究的问题不是 capacity 是否足够，而是 working set 是否保持高信噪比。

2.1 Append-only 的三个失败机制

- Context saturation：每次搜索结果和网页观察都进入历史，token cost 与 step 数近似单调增长。
- Context rot：即使没有超过硬长度限制，过多旧观察、失败分支和重复证据也会稀释当前约束，使模型更容易遗忘目标或重复搜索。
- Irreversible attention tax：模型在每一步都要重新“读”大量已经解决或无关的信息；这不是存储问题，而是推理带宽问题。

2.2 为什么整体摘要不够

周期性 summary 只有一种信息保真度：把历史全部重新生成成一段抽象文本。它无法表达“这段数字必须逐字保留”“这个分支应完全删除”“回到某个决策点重新探索”等不同需求。更重要的是，摘要通常由长度阈值触发，而不是由任务状态触发。

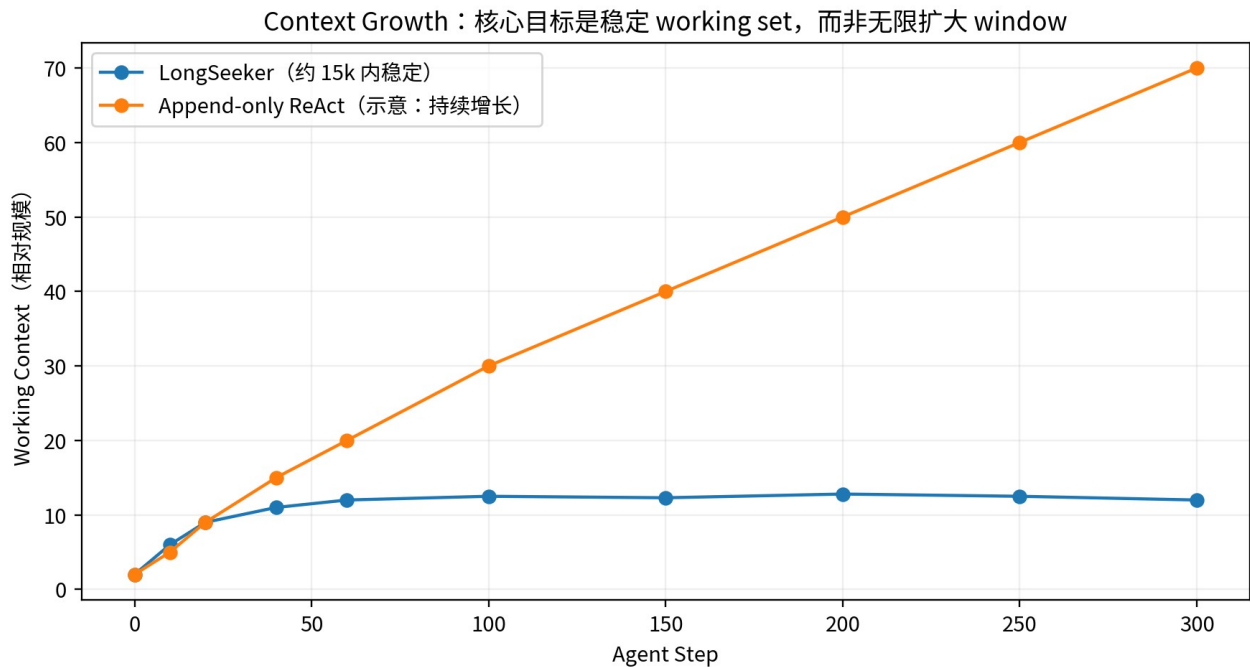


图3 真正的目标：让 Working Context 稳定，而不是只追求更大的最大窗口

3. Context-ReAct：把 Context Editing 升格为 Agent Action

标准 ReAct 的状态转移是 append-only: $H_t = [S_1, \dots, S_t]$ 。Context-ReAct 在 reasoning 与标准工具调用之间插入 M_t ，并先执行 $T(H_t, M_t)$ 得到 H'_t ，再进入下一步。

这意味着“什么时候编辑、编辑哪一段、用什么粒度编辑”都成为策略的一部分。论文强调 meta-operations 与 reasoning、tool call 在单次 autoregressive generation 中共同输出，而不是由外部 orchestration 代码基于 token 阈值强制触发。

3.1 四字段输出协议

字段	作用	为何重要
think	任务推理与未解决约束	给 meta-action 提供语义依据
meta_tool_call	一组 Context 操作及参数	修改 Working Context
motivation	解释为什么这次编辑/搜索有价值	显式训练决策理由
standard_tool_call	search_web / visit_web	继续与外部环境交互

这种结构的工程意义在于：Context Manager 不再是“后台清洁剂”，而是 Agent 决策环的一等公民。

4. 五种原子操作与信息保真度

五种原子 Meta-Operations：状态依赖的“信息保真度”

操作	语义	适合场景	工程价值
SKIP	不编辑	当前上下文已足够紧凑	零额外生成
COMPRESS	抽象压缩任意连续区间	已解决子问题/冗长轨迹	高压缩、可能丢细节
ROLLBACK	回退到较早步骤并记录原因	死胡同/错误分支	显式树搜索先验
SNIPPET	保留原文精确片段	数字、实体、URL、代码	无生成失真
DELETE	删除整步	失败/重复检索	直接降噪

图 4 五种操作对应五种不同的信息保真度与搜索状态

4.1 Skip

不修改上下文。它很重要，因为过度压缩同样有害；当当前状态仍然新鲜、紧凑、可用时，最优动作应当是零操作。

4.2 Compress

对任意连续历史区间做抽象摘要，不局限于最近窗口。这个“retroactive compress”允许 Agent 在后续获得更强证据后，回头把早期试探性搜索压成一句结论。

4.3 Rollback

回退到早期步骤并删除后续分支，同时保留“为什么回退”和可迁移的教训。它把树搜索/DFS 的 backtracking 先验显式加入 Context 管理。

4.4 Snippet

从某一步 observation 中按 anchor 精确保留原始 substring。与生成式摘要不同，Snippet 适合数字、实体名、URL、代码、关键原句等 precision-critical evidence，可避免摘要幻觉。

4.5 Delete

彻底删除无信息价值的一步，适合失败搜索、重复结果、错误工具输出。它不是“压缩成无内容”，而是给模型一个非常清晰的“这一步没有残余价值”的动作先验。

5. Expressive Completeness：为什么 Compress 理论上已足够但工程上不够

论文给出一个简单但重要的结论：如果 Compress 可以把整个历史替换成任意字符串，那么它理论上可以模拟 Skip、Rollback、Snippet 和 Delete，因此操作集合是 expressively complete。

但“可表达”不等于“容易学、低成本、低错误率”。专门操作提供 inductive bias：Rollback 明确告诉模型这是失败分支回退；Snippet 是 pointer-like 的无损抽取；Delete 是零生成的结构删除。

- Know-why：Agent action space 的设计不只追求数学完备性，还要追求“模型可学性”。
- Know-how：把高频且错误代价高的 context transformations 设计成独立 primitive，而不是让模型每次都自由文本 summary 模拟。
- Know-how：区分 lossy transformation（Compress）与 lossless transformation（Snippet / structural delete），这对 citation、数字、代码尤其关键。

6. 数据合成与训练：如何教会 Agent 管 Context

训练关键：监督的不只是答案，而是“中间 Context 编辑决策”

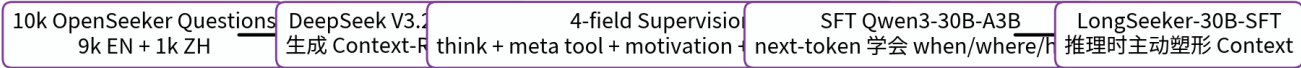


图 5 10k Context-ReAct 轨迹的监督生成流程

训练数据的真正新变量，是“中间 context management decision”。普通 search trajectory 只有 query、tool result、answer；LongSeeker 需要 teacher 在每个 step 同时标注 meta action、motivation 与下一次 tool call。

环节	论文设置	工程含义
Seed questions	OpenSeeker 10,000; 9,000 英文 + 1,000 中文	需要足够长、多跳、会产生上下文压力的问题
Teacher rollout	DeepSeek V3.2	让强模型示范 context editing
结构	think / meta_tool_call / motivation / standard_tool_call	把 Context Policy 显式序列化
Student	Qwen3-30B-A3B	与 Tongyi DeepResearch 同一规模级别便于比较
优化	SFT / next-token prediction	当前论文尚未用 RL 优化 meta action

论文明确把“未来用 RL 优化 meta-operation usage”列为下一步。这意味着 LongSeeker 更像 Context Policy 的 SFT proof-of-concept，而不是最终训练范式。

7. 推理 Runtime 与官方开源实现

截至 2026-08-13，官方 PolarSeeker/LongSeeker 仓库提供 inference + evaluation code。README 将运行方式描述为 separate-turn：每一步主模型输出 motivation、可选 meta tools、标准 tool call；Context 先被重写，再执行 search_web / visit_web。

模块	当前开源实现	作用
main_separate.py	异步多 worker inference	主 Agent loop
context.py	维护 Previous Steps	Working Context 数据结构
tools/meta_tool/	skip/compress/rollback/snippet/delete	Context Action Plane
search_web	Serper	Google 搜索
visit_web	Jina Reader + Summary LLM	网页抓取与证据摘要
eval.py	LLM-as-judge	答案正确性与成本统计

论文实验将 max tool call 设为 300；当前仓库 CLI 默认 tool_count_max=30，但可显式设置 300。这个差异很重要：默认值是工程运行参数，不应误当成论文评测预算。

- 值得迁移：Context state 与 full trajectory 分离保存。Working context 可以被编辑，但完整 trajectory 应保留用于审计、debug、训练和 replay。
- 值得迁移：visit_web 使用独立 Summary LLM，说明“Context management policy”和“网页内容压缩”可以是不同模型/不同成本层。

8. 实验结果：性能、Context Growth 与操作分布

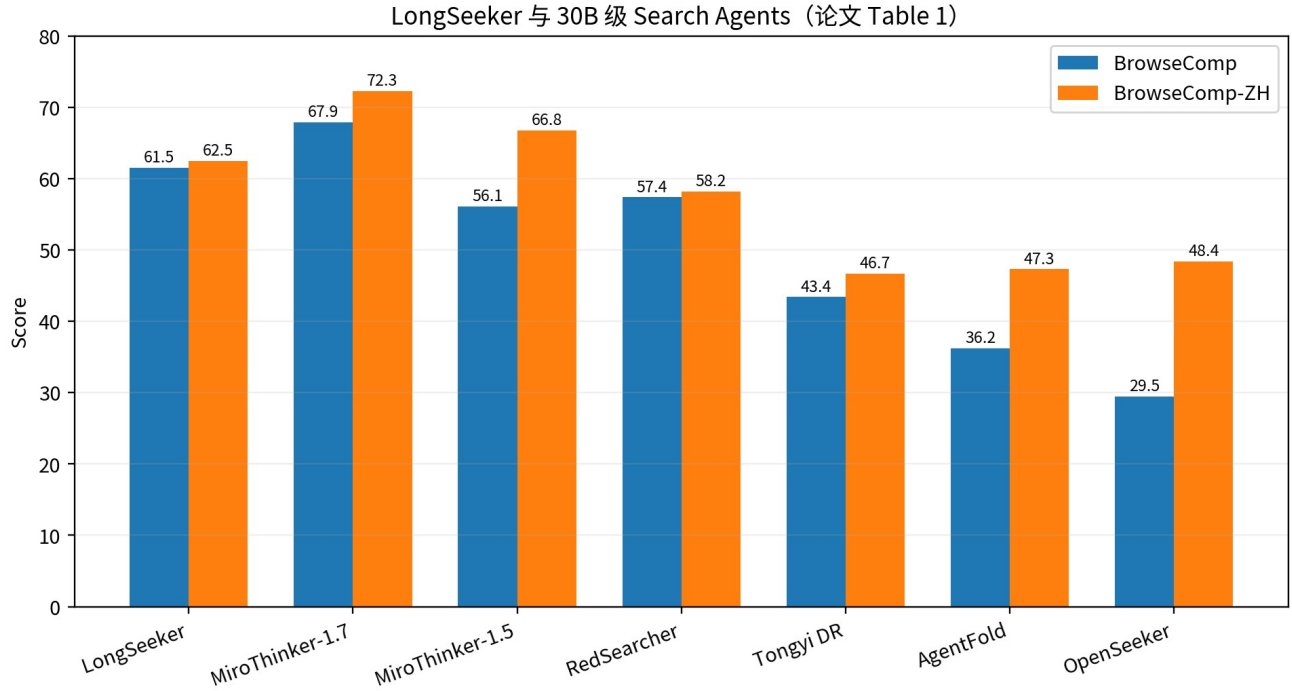


图 6 LongSeeker 与主要 30B 级 Search Agent 的 BrowseComp 对比

LongSeeker 在 BrowseComp/BrowseComp-ZH 上显著高于 Tongyi DeepResearch、AgentFold、OpenSeeker 与 IterResearch，但仍低于 MiroThinker-1.7-mini。重要的是，LongSeeker 只使用 SFT，而部分更强基线使用 CPT + SFT + RL。

Benchmark	LongSeeker	关键解释
BrowseComp	61.5	困难开放 Web 多跳检索
BrowseComp-ZH	62.5	中文长程检索
xbench-2505	78.0	职业对齐的 Deep Research 任务
GAIA-text	77.7	通用工具使用与多步推理

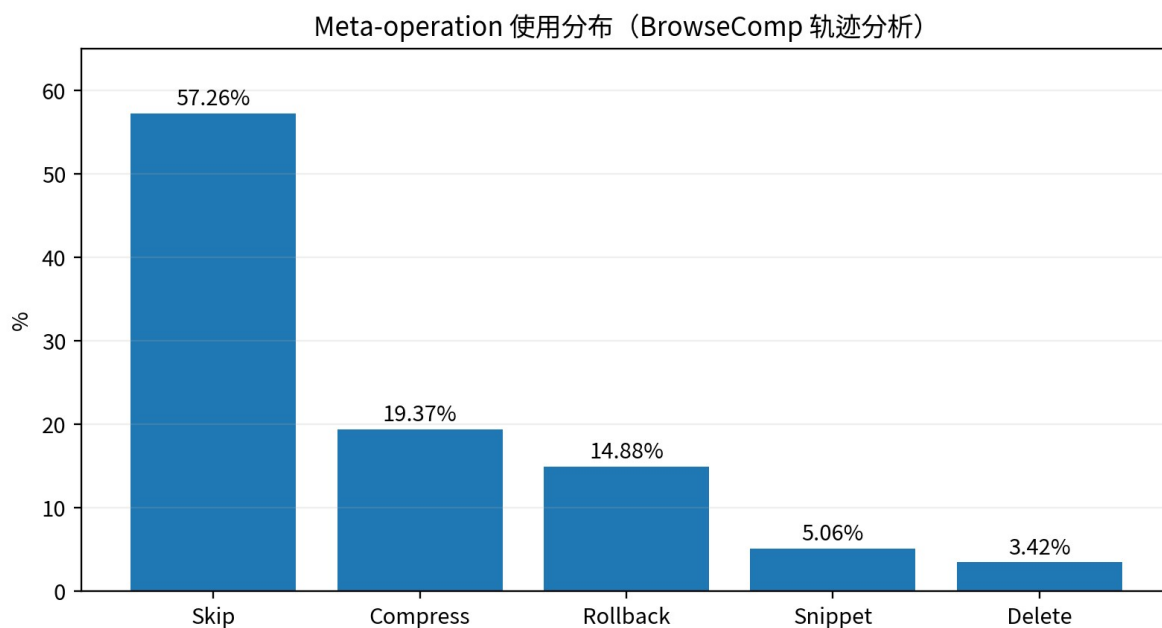


图 7 五种 Meta Action 都被模型实际使用；Skip 占比最高，剪枝较保守

操作分布约为 Skip 57.26%、Compress 19.37%、Rollback 14.88%、Snippet 5.06%、Delete 3.42%。作者解释 Snippet/Delete 较少，可能因为长程搜索早期很难确认哪些信息真正无用，因此模型倾向保守保留。

9. 为什么优于周期摘要 / Discard-all

论文在统一 DeepSeek-V3.2 base model 与相同配置下，将 Context-ReAct 与 Summary、Discard-all 做 controlled comparison。Figure 5 显示在相同步数预算下，Context-ReAct 始终更高。

策略	触发方式	保真度	主要失败模式
Summary	Context overflow 后	单一：生成式摘要	过度抽象、累积 summary error
Discard-all	达到阈值后	极低：清空历史	丢失进度与约束
ReSum	阈值/周期	整体 restartable summary	比 discard 好，但粒度仍粗
Context-ReAct	任务状态驱动	多分辨率	Policy 学错编辑动作时会损失信息

最值得提炼的 know-why：上下文管理的关键变量不是“压缩率”，而是 state-dependent fidelity。不同历史片段在不同阶段需要不同表示：原文、snippet、summary、删除或结构回退。

10. 与 ReSum / AgentFold / AdaCoM / ACM 的关系

Long-Horizon Context Management：从“压缩算法”转向“可学习的 Context Policy”

方法	谁决定何时编辑	Context Action	精细度	训练对象
ReSum	外部阈值/周期	整体摘要 + restart	低	无需训练主 Agent
AgentFold	Agent 主动	折叠历史子任务	中	SFT
Context-ReAct	Agent 主动逐步	5 种原子编辑	高	SFT 10k
AdaCoM	外部 Context LLM	灵活修改 frozen Agent 上下文	高	Context Manager RL
ACM (2026-07)	Agent 主动 + 外存	编辑 + offload + retrieve	高	后训练

图 8 Context Management 正从固定机制变成独立的可学习策略层

ReSum 的强项是极小侵入：周期或阈值触发，将历史压成可 restart 的 summary；LongSeeker 的强项是 context action 更细、触发由 Agent 语义状态决定。AgentFold 已经提出主动 folding，但 Context-ReAct 把操作空间进一步拆成五种 primitives，并支持向更早历史做精确编辑。

AdaCoM（2026-05）走了另一条非常重要的路线：冻结主 Agent，训练一个外部 Context Manager LLM 用 RL 修改它的上下文。这对闭源模型很有意义，因为不需要重新训练 Agent 本身。AdaCoM 还报告 Fidelity-Reliability Trade-off：更强的 Agent 倾向从高保真 Context 中获益，更弱的 Agent 反而可能需要更激进压缩。

ACM（2026-07）进一步加入外部 memory：被移除的内容不是永久消失，而是 offload 到长期存储，必要时按需 retrieve。这暴露了 LongSeeker 当前的一个缺口：Delete / Rollback 是工作区编辑，但没有内建“可召回的冷存储”。

11. 可迁移 Know-how：生产级 Research Agent 怎么设计

11.1 把 Context 当 Working Set，不当日志

完整 trajectory 是 audit log；Working Context 是推理时的工作集。两者必须分离。Agent 可以删除 Working Context 中的内容，但不能删除 Evidence Store / Trace Store 中的原始事实。

11.2 Context Action Plane

- keep/skip：无动作也是动作，避免过度压缩。
- compress：针对已解决子问题，把过程变成 conclusion + evidence refs + unresolved gaps。
- snippet/pin：对数字、引用、URL、代码做 lossless pin。
- delete：只删除低价值 working copy；原始对象仍留在 cold store。
- rollback：回退 search plan / hypothesis state，同时保留 failure reason，防止循环。

11.3 Evidence Store 与 Context Store 必须解耦

一个更安全的生产版本不应让 Compress 直接生成“事实本体”。更好的做法是 Summary 只保存 claim IDs / evidence IDs，真正原文仍可 retrieval-on-demand。这样可以兼得 LongSeeker 的紧凑 working context 和 PaperQA2/CaRR 的 provenance。

11.4 给 Context Policy 独立做 Eval

- Context token peak / average。
- 关键证据 survival rate：最后回答需要的 evidence 是否被误删。
- rollback precision：回退是否真的避免了失败分支。
- compression factuality：摘要是否保持原证据含义。
- tool-call redundancy：上下文整理后重复搜索是否下降。

12. 局限、风险与改进方向

- SFT imitation ceiling: teacher 的 Context Policy 会成为 student 的上限；论文没有用 rejection sampling 或 RL 直接优化 context action。
- Meta action 自身有错误代价：错误 Delete/Rollback 可能比保留噪声更糟，因为信息损失是结构性的。
- 没有外部可召回 memory：被删除的信息若后续变得重要，只能重新搜索；ACM/AdaCoM 的外部管理器路线值得融合。
- Benchmark 代表性：BrowseComp 与 BrowseComp-ZH 各采样 200 题，且部分评测设置包含 discard-all + 最多 5 rounds，需要谨慎比较不同论文的运行协议。
- 开放性：论文是 2026-05 arXiv preprint，当前结论应视为预印本结果，而非已完成同行评审的定论。

12.1 推荐改进：Context-ReAct 2.0

把 meta-action 从纯 SFT 升级成 cost-aware RL：reward 同时考虑 final correctness、evidence retention、context cost、重复工具调用、citation correctness 和 rollback quality。再增加 Archive/Retrieve 两个 lossless memory actions，形成 Hot Working Context + Cold Evidence Store。

13. 统一架构：Context Policy × Evidence State × Search Policy

结合前面的 Deep Research 系列，可以把下一代 Research Agent 写成三套耦合但可独立评测的 Policy：

Policy	核心状态	动作	代表系统
Search Policy	Question / Coverage / Uncertainty	search, visit, follow-up	STORM, MindSearch, WebThinker, Tongyi
Context Policy	Working history / relevance / fidelity need	compress, snippet, rollback, delete, retrieve	ReSum, AgentFold, LongSeeker, AdaCoM
Evidence Policy	claims / sources / provenance	store, rerank, verify, cite	PaperQA2, CaRR, DR Tulu

于是一个更完整的循环不再只是 $q_{t+1} = \text{Policy}(\text{question}, \text{evidence})$ ，而是：Search Policy 产生新证据；Evidence Policy 判断其可信度与结构；Context Policy 决定下一步推理需要看到哪些状态以及以何种保真度看到。

最终抽象：Deep Research Reliability $\approx$ Search Quality $\times$ Evidence Integrity $\times$ Context State Quality $\times$ Verifier Quality。任何一项接近 0，长程能力都会坍塌。

14. 结论

LongSeeker 最重要的意义，是把 Context Engineering 从“Host 侧的启发式清理规则”推进为“Agent 可学习、可组合、可解释的动作空间”。与 ReSum 相比，它不再等 Context 满了才总结；与 AgentFold 相比，它提供更细的 context primitives；与 AdaCoM 相比，它选择把 Context Policy 内生到 Agent 本身。

对工程实践最值得沉淀的一句话是：不要让 Agent 的 context 等同于它的历史。历史是日志，context 是工作集；工作集必须随任务状态变化，以不同保真度动态重构。

推荐的 2026 架构方向：Agent 内部保留轻量 Context-ReAct primitives；外部维护不可变 Evidence/Trace Store；当主模型不可训练时，用 AdaCoM 式外部 Context Manager；再用 RL 对 correctness - cost - evidence retention 做联合优化。

参考资料

- [1] Lu, Y. et al. LongSeeker: Elastic Context Orchestration for Long-Horizon Search Agents. arXiv:2605.05191, 2026.
- [2] PolarSeeker/LongSeeker official GitHub repository and README, accessed 2026-08-13.
- [3] Wu, X. et al. ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization. arXiv:2509.13313, 2025.
- [4] Ye, R. et al. AgentFold: Long-Horizon Web Agents with Proactive Context Management. arXiv:2510.24699, 2025.
- [5] Yao, Y. et al. ARC: Active and Reflection-driven Context Management for Long-Horizon Information Seeking Agents. arXiv:2601.12030, 2026.
- [6] Yi, L. et al. Learning Agent-Compatible Context Management for Long-Horizon Tasks (AdaCoM). arXiv:2605.30785, 2026.
- [7] Li, X. et al. ACM: Agentic Context Management for Long Horizon Tasks. arXiv:2607.23809, 2026.
- [8] Sun, W. et al. Scaling Long-Horizon LLM Agent via Context-Folding. arXiv:2510.11967, 2025.
- [9] Tongyi DeepResearch Technical Report / official repository, 2025-2026.

注：本文中的 LongSeeker 具体实验数字来自 arXiv v1；“2026 统一架构/生产改造建议”属于基于多篇工作的工程综合与推演，不是 LongSeeker 原论文原话。